Université Djillali Liabès de Sidi Bel Abbès

Faculté des Sciences Exactes

Département d'Informatique

Rapport du TP N°1

Algorithme d'Euclide étendu et Inversion modulaire

Réalisé par : Belhaouas Nesrine

Groupe : 01 ISI

Année universitaire : 2025 – 2026

24 février 2026

Table des matières

Introduction **2**

1 Objectifs du TP **3**

2 Rappel théoriques **4**
2.1 Identité de Bézout 4
2.2 Algorithme d'Euclide étendu 4
2.3 Inversion modulaire 5
2.3.1 Calcul par Euclide étendu 5

3 Application : Chiffrement affine **7**
3.1 Principe 7
3.2 Déchiffrement 7
3.3 Vérifier expérimentalement 7

Conclusion **9**

Introduction

Ce TP a pour objectif de familiariser l'étudiant avec l'algorithme d'Euclide étendu et l'inversion modulaire, deux outils essentiels pour de nombreux algorithmes de cryptographie. L'algorithme d'Euclide étendu permet non seulement de calculer le plus grand commun diviseur (PGCD) de deux entiers, mais également de déterminer les coefficients de Bézout, nécessaires pour trouver un inverse modulaire. L'inversion modulaire constitue la base du déchiffrement dans plusieurs méthodes classiques, dont le chiffrement affine.

Chapitre 1

Objectifs du TP

Ce tp permet : • d'Implémenter correctement l'algorithme d'Euclide étendu sur de grands entiers ; • Calculer des inverses modulaires en utilisant une implémentation maîtrisée ; • Exploiter l'inversion modulaire dans la construction d'un chiffrement classique ; • Comprendre les contraintes mathématiques garantissant la réversibilité d'un chiffrement.

Chapitre 2

Rappel théoriques

Soient a, b Z, non tous deux nuls. Le plus grand commun diviseur de a et b, noté gcd(a, b), est le plus grand entier positif divisant simultanément a et b.

2.1 Identité de Bézout

Il existe toujours des entiers relatifs u et v tels que :
au + bv = gcd(a, b).
L'algorithme d'Euclide étendu permet de calculer le PGCD de a et b et les coefficients u et v de bézout .

2.2 Algorithme d'Euclide étendu

L'algorithme d'Euclide étendu est une extension de l'algorithme d'Euclide classique qui maintient, à chaque itération, une représentation linéaire du reste courant en fonction des valeurs initiales.

Listing 2.1 – Pseudo-code de l'algorithme d'Euclide étendu

```
def extendedGCD(a, b):
    r, r1 = a, b
    u, u1 = 1, 0
    v, v1 = 0, 1
    while r1 != 0:
        q = r // r1
        (r, r1) = (r1, r - q * r1)
        (u, u1) = (u1, u - q * u1)
        (v, v1) = (v1, v - q * v1)
    return (r, u, v)
```

Le résultat (r, u, v) vérifie : $r = \gcd(a, b) = a \cdot u + b \cdot v$.

2.3 Inversion modulaire

Soient a, n Z avec n 2. On dit que a est inversible modulo n s'il existe x Z tel que :

ax=1 (mod n).

2.3.1 Calcul par Euclide étendu

L'inverse modulaire de a modulo n existe si et seulement si gcd(a, n) = 1. Dans ce cas, l'algorithme d'Euclide étendu appliqué à (a, n) fournit :

au + nv = 1, et la valeur u mod n constitue l'inverse modulaire recherché.

```python
class BigIntMath:
    def extendedGCD(self, a, b):
        r = a
        r2 = b
        u = 1
        v = 0
        u2 = 0
        v2 = 1

        while r2 != 0:
            q = r // r2
            r, r2, u, u2, v, v2 = (
                r2,
                r - q * r2,
                u2,
                u - q * u2,
                v2,
                v - q * v2
            )

        return r, u, v

    def modInverse(self, a, n):

        r, u, v = self.extendedGCD(a, n)

        if r != 1:
            print(f"Erreur : {a} n'a pas d'inverse modulo {n}")
            return None
        else:
```

FIGURE 2.1 –

```python

    def modInverse(self, a, n):

        r, u, v = self.extendedGCD(a, n)

        if r != 1:
            print(f"Erreur : {a} n'a pas d'inverse modulo {n}")
            return None
        else:
            x = u % n
            return x

math = BigIntMath()

a = int(input("Donner a : "))
b = int(input("Donner n (modulo) : "))

inverse = math.modInverse(a, b)

if inverse is None:
    print(f"a = {a} n'est pas inversible modulo {b}")
else:
    print(f"L'inverse de {a} mod {b} est {inverse}")
    print(f"Vérification : ({a} * {inverse}) % {b} = {(a * inverse) % b}")
```

FIGURE 2.2 –

```
PROBLEMS 4    OUTPUT    DEBUG CONSOLE

PS C:\Users\LIBERTY\AppData\Local\Pro
tp1crypto.py
Donner a : 5
Donner n (modulo) : 26
L'inverse de 5 mod 26 est 21
Vérification : (5 * 21) % 26 = 1
```

FIGURE 2.3 – l'execution

Chapitre 3

Application : Chiffrement affine

3.1 Principe

Le chiffrement affine est défini par :

$$E_{a,b}(m) = (a \cdot m + b) \bmod n$$

avec la condition $\gcd(a, n) = 1$.

3.2 Déchiffrement

Le déchiffrement repose sur l'inverse modulaire de a :

$$D_{a,b}(c) = a^{-1} \cdot (c - b) \bmod n$$

3.3 Vérifier expérimentalement

Da,b(Ea,b(m)) = m.

```python
class AffineCipher:
    def __init__(self,a,b,n):
        self.a= a
        self.b = b
        self.n = n

    def encryptcipher(self, message):
        return (self.a * m + self.b) % self.n
    def decrypt(self, k: int):
        return (self.a_inv * (k - self.b)) % self.n


if __name__ == "__main__":
    a = int(input("Donner la clé a : "))
    b = int(input("Donner la clé b : "))
    n = 26

    cipher = AffineCipher(a, b, n)
```

FIGURE 3.1 – la clasee affinecipher

```python
for I in range(len(n)):
    c = cipher.encrypt(m)
    m_recov = cipher.decrypt(c)
    print(f"m={m:2d} → E(m)={c:2d} → D(E(m))={m_recov:2d}")
    if m != m_recov:
        ok = False

if ok:
    print("\n Résultat : D(E(m)) = m pour tous les m ∈ [0, n-1]")
else:
    print("\n Erreur : la propriété ne vérifie pas pour certains m")
```

FIGURE 3.2 – code de vérification

Conclusion

Ce travail a permis de comprendre le lien fondamental entre les mathématiques de base et la cryptographie. L'algorithme d'Euclide étendu et l'inversion modulaire constituent la base de nombreux algorithmes modernes, notamment le chiffrement RSA. Le chiffrement affine illustre concrètement la nécessité d'une bonne compréhension de ces outils pour assurer la sécurité et la réversibilité des systèmes de chiffrement.

1/2